

Disciplina: DCA 3703 - PROGRAMAÇÃO PARALELA

Professor: Samuel Xavier De Souza

Nome: José Felix Rodrigues Anselmo

Tarefa 11 - PARTICIONAMENTO DE DADOS E BALANCEAMENTO DECARGA

1 - Enunciado

Escreva um código que simule o movimento de um fluido ao longo do tempo usando a equação de Navier-Stokes, considerando apenas os efeitos da viscosidade. Desconsidere a pressão e quaisquer forças externas. Utilize diferenças finitas para discretizar o espaço e simule a evolução da velocidade do fluido no tempo. Inicialize o fluido parado ou com velocidade constante e verifique se o campo permanece estável. Em seguida, crie uma pequena perturbação e observe se ela se difunde suavemente. Após validar o código, paralelize-o com OpenMP e explore o impacto das cláusulas `schedule` e `collapse` no desempenho da execução paralela.

2 - Metodologia

Como o enunciado requer um pouco do entendimento sobre a equação de Navier-Stokes para que seja possível entender a execução do código em C.

2.1 - O que é a equação de Navier-Stokes:

As equações de Navier-Stokes formam o conjunto fundamental de equações diferenciais parciais não lineares que descrevem o movimento de fluidos (líquidos e gases). Elas aplicam as leis de Newton da mecânica (especificamente a conservação do momento linear e da massa) a meios contínuos.

Componentes Principais da Equação de Momento Para um fluido incompressível com densidade constante, a formulação geral para o vetor velocidade $\mathbf{u}$ é expressa como:

$$\rho \left(\frac{\partial \mathbf{u}}{\partial t} + \mathbf{u} \cdot \nabla \mathbf{u} \right) = -\nabla p + \mu \nabla^2 \mathbf{u} + \mathbf{f}$$

Figura 1: Equação do fluido incompressível

Cada termo representa uma força física atuando sobre o fluido:

- $\partial_t \mathbf{u}$: Aceleração local (como a velocidade muda com o tempo em um ponto fixo).
- $\rho(\mathbf{u} \cdot \nabla \mathbf{u})$: **Advecção (ou convecção)**, que representa o transporte de momento pelo próprio fluxo do fluido (termo não linear responsável pela complexidade e turbulência).
- $-\nabla p$: **Gradiente de pressão**, as forças geradas por diferenças de pressão no fluido.
- $\mu \nabla^2 \mathbf{u}$: **Termo viscoso** (difusão de momento), correspondente ao atrito interno do fluido (o único efeito isolado e simulado no código de diferenças finitas anterior).
- $\mathbf{f}$: Forças externas volumétricas, como a gravidade ou forças eletromagnéticas.

Equação da Continuidade Além do momento, as simulações de fluidos exigem a conservação da massa. Para fluidos incompressíveis, ela se resume a:

$$\nabla \cdot \mathbf{u} = 0$$

Isso significa que a divergência do campo de velocidades deve ser nula em qualquer ponto (o que entra em um volume deve sair imediatamente).

Aplicações e Desafios Matemáticos As aplicações cobrem a previsão do tempo, aerodinâmica de aeronaves, projeto de turbinas, hidrodinâmica naval e computação gráfica para simulação de fumaça e água.

Do ponto de vista teórico, as equações de Navier-Stokes constituem um dos sete **Problemas do Prêmio do Milênio** do Clay Mathematics Institute. Até hoje, matemáticos não conseguiram provar formalmente se, dadas condições iniciais suaves em 3D, sempre existirão soluções suaves para todo o tempo futuro, ou se o sistema pode desenvolver singularidades (como infinitos locais).

Após entender como o fluido se comporta na prática, o próximo passo foi criar uma implementação em C na qual fosse possível em primeiro lugar gerar os dados necessários para utilizar uma implementação em python para a visualização dos dados do experimento.

```
gcc navier_stokes.c -o navier -lm
./navier
```

Após verificar a versão sequencial está foi implementado com base na mesma, adicionando as diretivas do omp para a versão paralela que foi executada como está descrito acima usando o schedule e o collapse.

```
gcc -fopenmp navier_stokes_p.c -o navier_stokes_collapse2 -lm
```

```
export OMP_NUM_THREADS=8; time ./navier_stokes_collapse2
```

2.2 - Particionamento de Dados e Balanceamento de Carga

O particionamento (ou decomposição) é uma das primeiras etapas no projeto de um programa paralelo e consiste em dividir o trabalho global da aplicação em partes menores (tarefas) que serão distribuídas entre os diferentes fluxos de execução (threads ou processos). O objetivo é extrair o máximo de paralelismo mantendo a sobrecarga de comunicação e sincronização mais baixa do que o trabalho útil realizado. Existem duas abordagens básicas:

- Decomposição do domínio de dados: o foco está nos dados manipulados pela aplicação. O conjunto de dados (como um vetor ou matriz) é fatiado entre as tarefas, em blocos (fatias contíguas), de forma cíclica (round-robin) ou bloco-cíclica.
- Decomposição funcional: o foco está no tipo de computação a ser realizada. O problema é decomposto de acordo com as diferentes funções ou etapas do trabalho global.

O balanceamento de carga é a estratégia de distribuir as tarefas de maneira homogênea entre os processadores disponíveis, com o objetivo de mantê-los todos ocupados durante todo o tempo de execução — um processador ocioso enquanto outro ainda trabalha representa perda de desempenho e eficiência. Ele se divide em duas categorias:

- Balanceamento estático: as tarefas são atribuídas aos processadores uma única vez, no início do processamento, com base apenas nas dimensões das estruturas de dados (por exemplo, N/P iterações para cada um dos P processadores). É ideal quando o tempo de computação por item é constante e previsível de antemão.
- Balanceamento dinâmico: a carga é ajustada e atribuída em tempo de execução, conforme os processadores concluem suas tarefas e ficam livres. É indispensável para problemas irregulares, em que o tempo de execução varia conforme o conteúdo dos dados. Exemplo clássico: o algoritmo de saco de tarefas (mestre/trabalhador), em que um processo mestre mantém um repositório de tarefas e envia novas parcelas de trabalho aos trabalhadores assim que estes ficam ociosos.

Essas duas ideias mapeiam diretamente para as cláusulas do OpenMP avaliadas neste trabalho: `schedule(static)` implementa balanceamento estático (divisão em blocos, decidida

uma única vez); `schedule(dynamic)` e `schedule(guided)` implementam balanceamento dinâmico (o mesmo princípio do saco de tarefas, com uma fila de trabalho compartilhada); e `collapse` atua sobre a decomposição do domínio, fundindo dimensões da malha para aumentar o número de fatias disponíveis para distribuir entre as threads.

3 - Código Fonte

Para a versão sequencial foi usado o seguinte código, que foi gerado com ajudar de uma IA como recomendado pelo professor durante a aula pois para esse exemplo não seria necessário o conhecimento de mecânica dos fluídos e entendimento de navier-stokes:

3.1 - Código Navier Stokes(Sequencial)

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <time.h>

#define LX 1.0
#define LY 1.0
#define LZ 1.0
#define NU 0.01

#define AMPLITUDE 2.0
#define LARGURA 0.03

#define NX_PADRAO 64
#define NY_PADRAO 64
#define NZ_PADRAO 64
#define N_STEPS_PADRAO 200
#define CSV_PADRAO "difusao3d_sequencial.csv"
#define MAX_PTS_PADRAO 48
#define N_SNAPS_PADRAO 5

static double agora_em_segundos(void) {
    struct timespec ts;
    clock_gettime(CLOCK_MONOTONIC, &ts);
    return (double) ts.tv_sec + (double) ts.tv_nsec / 1e9;
}

static inline long idx(int i, int j, int k, int NY, int NZ) {
    return ((long) i * NY + j) * NZ + k;
}

static double *aloca_campo(int NX, int NY, int NZ) {
    double *campo = (double *) calloc((size_t) NX * NY * NZ, sizeof(double));
    if (campo == NULL) {
        fprintf(stderr, "Erro: falha ao alocar memoria.\n");
        exit(EXIT_FAILURE);
    }
}
```

```

    return campo;
}

static void aplica_contorno(double *c, int NX, int NY, int NZ) {
    for (int i = 0; i < NX; i++)
        for (int j = 0; j < NY; j++) {
            c[idx(i, j, 0, NY, NZ)] = 0.0;
            c[idx(i, j, NZ - 1, NY, NZ)] = 0.0;
        }
    for (int i = 0; i < NX; i++)
        for (int k = 0; k < NZ; k++) {
            c[idx(i, 0, k, NY, NZ)] = 0.0;
            c[idx(i, NY - 1, k, NY, NZ)] = 0.0;
        }
    for (int j = 0; j < NY; j++)
        for (int k = 0; k < NZ; k++) {
            c[idx(0, j, k, NY, NZ)] = 0.0;
            c[idx(NX - 1, j, k, NY, NZ)] = 0.0;
        }
}

static void passo_difusao(const double *c, double *cn, double dt,
                        double dx, double dy, double dz, int NX, int NY, int NZ) {
    const double kx = NU * dt / (dx * dx);
    const double ky = NU * dt / (dy * dy);
    const double kz = NU * dt / (dz * dz);

    for (int i = 1; i < NX - 1; i++) {
        for (int j = 1; j < NY - 1; j++) {
            for (int k = 1; k < NZ - 1; k++) {
                long p = idx(i, j, k, NY, NZ);
                double centro = c[p];
                cn[p] = centro
                    + kx * (c[idx(i + 1, j, k, NY, NZ)] - 2.0 * centro + c[idx(i - 1, j, k, NY, NZ)])
                    + ky * (c[idx(i, j + 1, k, NY, NZ)] - 2.0 * centro + c[idx(i, j - 1, k, NY, NZ)])
                    + kz * (c[idx(i, j, k + 1, NY, NZ)] - 2.0 * centro + c[idx(i, j, k - 1, NY, NZ)]);
            }
        }
    }
}

static void aplica_perturbacao_gaussiana(double *u, double dx, double dy, double dz,
                                         int NX, int NY, int NZ) {
    for (int i = 0; i < NX; i++)
        for (int j = 0; j < NY; j++)
            for (int k = 0; k < NZ; k++) {
                double x = i * dx - 0.5 * LX;
                double y = j * dy - 0.5 * LY;
                double z = k * dz - 0.5 * LZ;
            }
}

```

```

        double r2 = x * x + y * y + z * z;
        u[idx(i, j, k, NY, NZ)] += AMPLITUDE * exp(-r2 / (2.0 * LARGURA * LARGURA));
    }
    aplica_contorno(u, NX, NY, NZ);
}

static long grava_snapshot(FILE *f, const double *u, int NX, int NY, int NZ,
    double dx, double dy, double dz, long passo, double t,
    int ex, int ey, int ez) {
    long linhas = 0;
    for (int i = 0; i < NX; i += ex)
        for (int j = 0; j < NY; j += ey)
            for (int k = 0; k < NZ; k += ez) {
                fprintf(f, "%ld,%.6g,%d,%d,%d,%.5f,%.5f,%.5f,%.6g\n",
                    passo, t, i, j, k, i * dx, j * dy, k * dz,
                    u[idx(i, j, k, NY, NZ)]);
                linhas++;
            }
    return linhas;
}

int main(int argc, char **argv) {
    int NX = (argc > 1) ? atoi(argv[1]) : NX_PADRAO;
    int NY = (argc > 2) ? atoi(argv[2]) : NY_PADRAO;
    int NZ = (argc > 3) ? atoi(argv[3]) : NZ_PADRAO;
    long N_STEPS = (argc > 4) ? atol(argv[4]) : N_STEPS_PADRAO;
    const char *arq_csv = (argc > 5) ? argv[5] : CSV_PADRAO;
    int max_pts = (argc > 6) ? atoi(argv[6]) : MAX_PTS_PADRAO;
    int n_snaps = (argc > 7) ? atoi(argv[7]) : N_SNAPS_PADRAO;

    if (NX < 3 || NY < 3 || NZ < 3 || N_STEPS < 1 || max_pts < 2 || n_snaps < 2) {
        fprintf(stderr, "Uso: %s [NX>=3] [NY>=3] [NZ>=3] [N_STEPS>=1] [saida.csv]
[MX_PTS>=2] [N_SNAPS>=2]\n", argv[0]);
        return EXIT_FAILURE;
    }

    double dx = LX / (NX - 1);
    double dy = LY / (NY - 1);
    double dz = LZ / (NZ - 1);
    double limite_estabilidade = 0.5 / (NU * (1.0 / (dx * dx) + 1.0 / (dy * dy) + 1.0 / (dz * dz)));
    double dt = 0.4 * limite_estabilidade;

    double *u = aloca_campo(NX, NY, NZ);
    double *u_novo = aloca_campo(NX, NY, NZ);
    long total_pontos = (long) NX * NY * NZ;

    aplica_perturbacao_gaussiana(u, dx, dy, dz, NX, NY, NZ);

    FILE *f = fopen(arq_csv, "w");

```

```

if (f == NULL) {
    fprintf(stderr, "Erro: nao foi possivel abrir '%s'.\n", arq_csv);
    return EXIT_FAILURE;
}
fprintf(f, "passo,t,i,j,k,x,y,z,u\n");

int ex = (NX + max_pts - 1) / max_pts;
int ey = (NY + max_pts - 1) / max_pts;
int ez = (NZ + max_pts - 1) / max_pts;

fprintf(stderr, "Malha = %d x %d x %d (%.2f milhoes de pontos) | passos = %ld |
SEQUENCIAL | dt = %.3e\n", NX, NY, NZ, total_pontos / 1e6, N_STEPS, dt);

double tempo_calculo = 0.0; /* so os passos; a gravacao do CSV fica de fora */
long linhas = 0;
int s = 0;

for (long passo = 0; passo <= N_STEPS; passo++) {
    /* snapshots igualmente espacados entre o passo 0 e o final */
    if (s < n_snaps && passo == (long) s * N_STEPS / (n_snaps - 1)) {
        linhas += grava_snapshot(f, u, NX, NY, NZ, dx, dy, dz, passo, passo * dt, ex, ey, ez);
        while (s < n_snaps && passo == (long) s * N_STEPS / (n_snaps - 1)) s++;
    }
    if (passo == N_STEPS) break;

    double inicio = agora_em_segundos();
    passo_difusao(u, u_novo, dt, dx, dy, dz, NX, NY, NZ);
    double *tmp = u; u = u_novo; u_novo = tmp; /* troca de ponteiros, sem memcpy */
    tempo_calculo += agora_em_segundos() - inicio;
}
fclose(f);

double soma = 0.0;
for (long p = 0; p < total_pontos; p++) soma += u[p];

fprintf(stderr, "(soma de u) = %.10f\n", soma);
fprintf(stderr, "csv salvo como: %s: %d snapshots, %ld linhas (amostragem %dx%dx%d)\n",
arq_csv, s, linhas, ex, ey, ez);
printf("Tempo: %.6f s\n", tempo_calculo);

free(u);
free(u_novo);
return 0;
}

```

3.2 - Código Navier Stokes (Paralelo)

A versão paralela foi construída sobre a sequencial, acrescentando diretivas OpenMP ao laço de atualização do campo. Foram geradas 11 variantes do mesmo código: `#pragma omp for schedule(static, N)`, com *schedule static*, *dynamic* e *guided*, cada um com *chunk N* = 2, 4 e 8, além de duas variantes com a cláusula `collapse(2)` e `collapse(3)`, que fundem laços aninhados em um único espaço de iterações a ser dividido entre as threads.

```
#define _POSIX_C_SOURCE 199309L
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <time.h>
#include <omp.h>

#define LX 1.0
#define LY 1.0
#define LZ 1.0
#define NU 0.01

#define AMPLITUDE 2.0
#define LARGURA 0.03

#define NX_PADRAO 64
#define NY_PADRAO 64
#define NZ_PADRAO 64
#define N_STEPS_PADRAO 200
#define CSV_PADRAO "difusao3d_p_.csv"
#define MAX_PTS_PADRAO 48
#define N_SNAPS_PADRAO 5

static double agora_em_segundos(void) {
    struct timespec ts;
    clock_gettime(CLOCK_MONOTONIC, &ts);
    return (double) ts.tv_sec + (double) ts.tv_nsec / 1e9;
}

static inline long idx(int i, int j, int k, int NY, int NZ) {
    return ((long) i * NY + j) * NZ + k;
}

static double *aloca_campo(int NX, int NY, int NZ) {
    double *campo = (double *) calloc((size_t) NX * NY * NZ, sizeof(double));
    if (campo == NULL) {
        fprintf(stderr, "Erro: falha ao alocar memoria.\n");
        exit(EXIT_FAILURE);
    }
    return campo;
}
```

```

}

static void aplica_contorno(double *c, int NX, int NY, int NZ) {
    for (int i = 0; i < NX; i++)
        for (int j = 0; j < NY; j++) {
            c[idx(i, j, 0, NY, NZ)] = 0.0;
            c[idx(i, j, NZ - 1, NY, NZ)] = 0.0;
        }
    for (int i = 0; i < NX; i++)
        for (int k = 0; k < NZ; k++) {
            c[idx(i, 0, k, NY, NZ)] = 0.0;
            c[idx(i, NY - 1, k, NY, NZ)] = 0.0;
        }
    for (int j = 0; j < NY; j++)
        for (int k = 0; k < NZ; k++) {
            c[idx(0, j, k, NY, NZ)] = 0.0;
            c[idx(NX - 1, j, k, NY, NZ)] = 0.0;
        }
}

static void passo_difusao(const double *c, double *cn, double dt, double dx, double dy, double
dz, int NX, int NY, int NZ) {
    const double kx = NU * dt / (dx * dx);
    const double ky = NU * dt / (dy * dy);
    const double kz = NU * dt / (dz * dz);

    #pragma omp parallel
    {
        #pragma omp for collapse(2)
        for (int i = 1; i < NX - 1; i++) {
            for (int j = 1; j < NY - 1; j++) {
                for (int k = 1; k < NZ - 1; k++) {
                    long p = idx(i, j, k, NY, NZ);
                    double centro = c[p];
                    cn[p] = centro
                        + kx * (c[idx(i + 1, j, k, NY, NZ)] - 2.0 * centro + c[idx(i - 1, j, k, NY, NZ)])
                        + ky * (c[idx(i, j + 1, k, NY, NZ)] - 2.0 * centro + c[idx(i, j - 1, k, NY, NZ)])
                        + kz * (c[idx(i, j, k + 1, NY, NZ)] - 2.0 * centro + c[idx(i, j, k - 1, NY, NZ)]);
                }
            }
        }
    }
}

/* Gaussian 3D no centro do dominio. */
static void aplica_perturbacao_gaussiana(double *u, double dx, double dy, double dz,

```

```

        int NX, int NY, int NZ) {
    for (int i = 0; i < NX; i++)
        for (int j = 0; j < NY; j++)
            for (int k = 0; k < NZ; k++) {
                double x = i * dx - 0.5 * LX;
                double y = j * dy - 0.5 * LY;
                double z = k * dz - 0.5 * LZ;
                double r2 = x * x + y * y + z * z;
                u[idx(i, j, k, NY, NZ)] += AMPLITUDE * exp(-r2 / (2.0 * LARGURA * LARGURA));
            }
    aplica_contorno(u, NX, NY, NZ);
}

/* Grava um snapshot (subamostrado com passo ex,ey,ez) no CSV. */
static long grava_snapshot(FILE *f, const double *u, int NX, int NY, int NZ,
                           double dx, double dy, double dz, long passo, double t,
                           int ex, int ey, int ez) {
    long linhas = 0;
    for (int i = 0; i < NX; i += ex)
        for (int j = 0; j < NY; j += ey)
            for (int k = 0; k < NZ; k += ez) {
                fprintf(f, "%ld,%d,%d,%d,%d,%d,%d,%d,%d,%d\n",
                        passo, t, i, j, k, i * dx, j * dy, k * dz,
                        u[idx(i, j, k, NY, NZ)]);
                linhas++;
            }
    return linhas;
}

int main(int argc, char **argv) {
    int NX = (argc > 1) ? atoi(argv[1]) : NX_PADRAO;
    int NY = (argc > 2) ? atoi(argv[2]) : NY_PADRAO;
    int NZ = (argc > 3) ? atoi(argv[3]) : NZ_PADRAO;
    long N_STEPS = (argc > 4) ? atol(argv[4]) : N_STEPS_PADRAO;
    const char *arq_csv = (argc > 5) ? argv[5] : CSV_PADRAO;
    int max_pts = (argc > 6) ? atoi(argv[6]) : MAX_PTS_PADRAO;
    int n_snaps = (argc > 7) ? atoi(argv[7]) : N_SNAPS_PADRAO;

    if (NX < 3 || NY < 3 || NZ < 3 || N_STEPS < 1 || max_pts < 2 || n_snaps < 2) {
        fprintf(stderr, "Uso: %s [NX>=3] [NY>=3] [NZ>=3] [N_STEPS>=1] [saida.csv]
[MAY_PTS>=2] [N_SNAPS>=2]\n", argv[0]);
        return EXIT_FAILURE;
    }

    double dx = LX / (NX - 1);
    double dy = LY / (NY - 1);
    double dz = LZ / (NZ - 1);
    double limite_estabilidade = 0.5 / (NU * (1.0 / (dx * dx) + 1.0 / (dy * dy) + 1.0 / (dz * dz)));
    double dt = 0.4 * limite_estabilidade;

```

```

double *u = aloca_campo(NX, NY, NZ);
double *u_novo = aloca_campo(NX, NY, NZ);
long total_pontos = (long) NX * NY * NZ;

aplica_perturbacao_gaussiana(u, dx, dy, dz, NX, NY, NZ);

FILE *f = fopen(arq_csv, "w");
if (f == NULL) {
    fprintf(stderr, "Erro: nao foi possivel abrir '%s'.\n", arq_csv);
    return EXIT_FAILURE;
}
fprintf(f, "passo,t,i,j,k,x,y,z,u\n");

int ex = (NX + max_pts - 1) / max_pts;
int ey = (NY + max_pts - 1) / max_pts;
int ez = (NZ + max_pts - 1) / max_pts;

fprintf(stderr, "Malha = %d x %d x %d (%.2f milhoes de pontos) | passos = %ld |
SEQUENCIAL | dt = %.3e\n", NX, NY, NZ, total_pontos / 1e6, N_STEPS, dt);

double tempo_calculo = 0.0; /* so os passos; a gravacao do CSV fica de fora */
long linhas = 0;
int s = 0;

for (long passo = 0; passo <= N_STEPS; passo++) {
    /* snapshots igualmente espacados entre o passo 0 e o final */
    if (s < n_snaps && passo == (long) s * N_STEPS / (n_snaps - 1)) {
        linhas += grava_snapshot(f, u, NX, NY, NZ, dx, dy, dz, passo, passo * dt, ex, ey, ez);
        while (s < n_snaps && passo == (long) s * N_STEPS / (n_snaps - 1)) s++;
    }
    if (passo == N_STEPS) break;

    double inicio = agora_em_segundos();
    passo_diffusao(u, u_novo, dt, dx, dy, dz, NX, NY, NZ);
    double *tmp = u; u = u_novo; u_novo = tmp; /* troca de ponteiros, sem memcpy */
    tempo_calculo += agora_em_segundos() - inicio;
}
fclose(f);

double soma = 0.0;
for (long p = 0; p < total_pontos; p++) soma += u[p];

fprintf(stderr, "(soma de u) = %.10f\n", soma);
fprintf(stderr, "[csv] %s: %d snapshots, %ld linhas (amostragem %dx%dx%d)\n", arq_csv, s,
linhas, ex, ey, ez);
printf("Tempo: %.6f s\n", tempo_calculo);

```

```

free(u);
free(u_novo);
return 0;
}

```

4 - Resultado

Os testes inicialmente foram feitos com o código sequencial como base de corretude para as alterações que seriam feitas subsequentemente para o cumprimento do pedido pela tarefa.

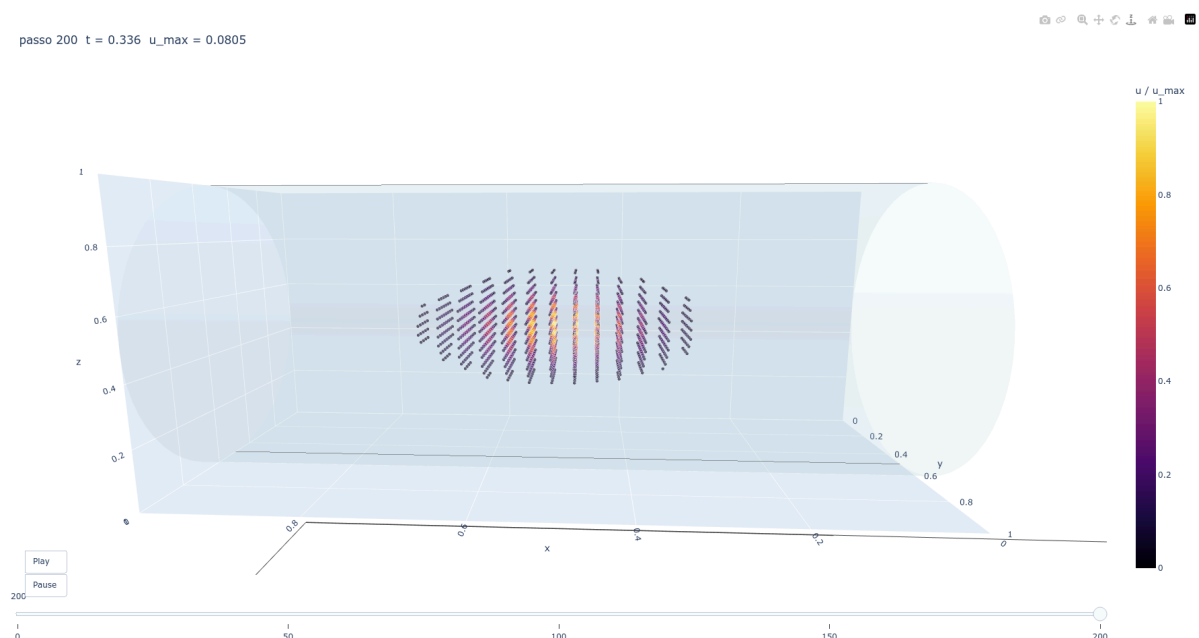


Figura 2: Visualização da saída em formato 3d

Na figura 2 podemos ver o comportamento das moléculas ao se deslocarem conforme o tempo passa e o nível de desordem aumenta. As imagens foram geradas por uma aplicação “visualiza.py” que não foi anexada pois serve apenas para pegar os .csv’s gerados e a partir deles plotar as imagens.

4.1 - Ambiente e metodologia dos testes paralelos

Os testes foram executados em um notebook IdeaPad 3 15ALC6 com Linux, compilando com o gcc e a flag `-fopenmp`. A simulação usou uma malha de 64 x 64 x 64 pontos (0,26 milhões), 200 passos de tempo e $dt = 1,680e-03$, gravando 5 snapshots em CSV (amostragem 2x2x2). O número de threads foi controlado pela variável de ambiente `OMP_NUM_THREADS` (2, 4 e 8), sem alterar o código, e o tempo considerado é o valor impresso pelo próprio programa. Foram avaliadas as 11 variantes descritas na seção 3.2

em cada número de threads, totalizando 33 configurações, cada uma executada uma vez. Como referência, a versão sequencial levou 1,009 s.

Componente	Especificação
Notebook	Lenovo IdeaPad 3 15ALC6
SO	Ubuntu 26.04 LTS
Kernel	Linux 7.0.0-29-generic
CPU	AMD Ryzen 5 5500U — 6C/12T
Arquitetura	x86_64
RAM	9,05 GiB
Desktop	GNOME 50.1
Shell	Zsh 5.9
Terminal	Kitty 0.45.0
Compilador	GCC

4.2 - Desempenho: schedule e collapse

A Tabela 2 reúne os tempos, o speedup e a eficiência das 33 configurações e a Figura 3 mostra as mesmas curvas. A soma de u foi 212,6596807633 em todas as execuções, igual à da versão sequencial, o que indica que a paralelização não alterou o resultado numérico.

Tabela 2: Tempo de execução, speedup e eficiência por variante e número de threads

Versão	2 threads			4 threads			8 threads		
	Tempo (s)	Speedup	Efic. (%)	Tempo (s)	Speedup	Efic. (%)	Tempo (s)	Speedup	Efic. (%)
static,2	0,631	1,60	80	0,347	2,91	73	0,300	3,37	42
static,4	0,612	1,65	82	0,348	2,90	72	0,274	3,68	46
static,8	0,541	1,87	93	0,299	3,37	84	0,267	3,77	47
dynamic,2	0,655	1,54	77	0,339	2,98	74	0,240	4,20	52
dynamic,4	0,567	1,78	89	0,315	3,20	80	0,270	3,74	47
dynamic,8	0,539	1,87	94	0,313	3,23	81	0,283	3,56	44

Versão	2 threads			4 threads			8 threads		
	Tempo (s)	Speedup	Efic. (%)	Tempo (s)	Speedup	Efic. (%)	Tempo (s)	Speedup	Efic. (%)
guided,2	0,523	1,93	96	0,319	3,16	79	0,260	3,88	48
guided,4	0,570	1,77	89	0,326	3,09	77	0,266	3,79	47
guided,8	0,531	1,90	95	0,337	2,99	75	0,281	3,59	45
collapse(2)	0,507	1,99	100	0,276	3,65	91	0,261	3,86	48
collapse(3)	0,559	1,81	90	0,298	3,39	85	0,255	3,95	49

Speedup $S = T_{seq}/T_{par}$ e eficiência $E = S/p$, com $T_{seq} = 1,009$ s (versão sequencial). Em negrito, o menor tempo para cada número de threads.

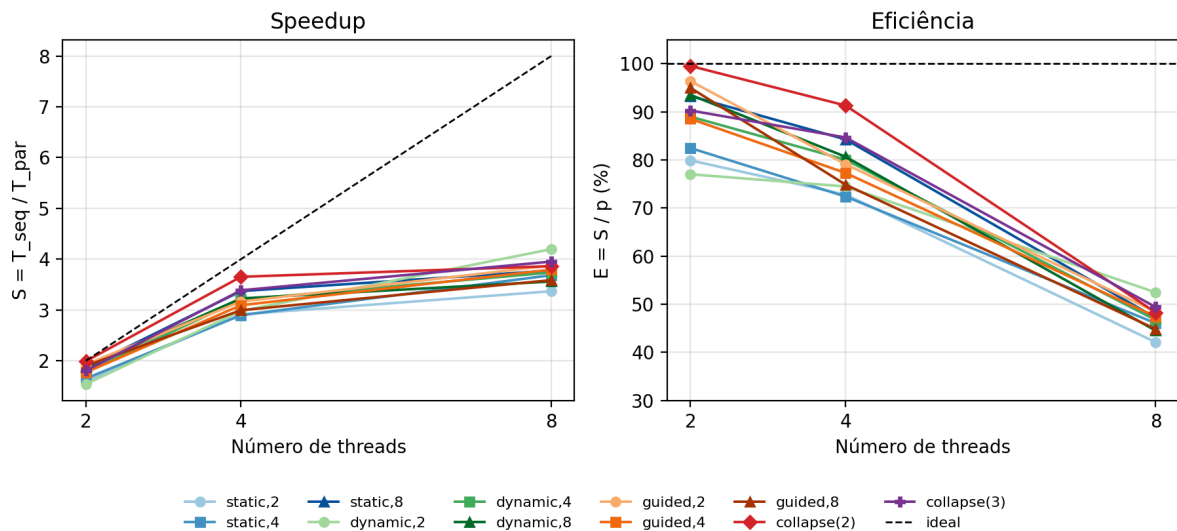


Figura 3: Speedup e eficiência em função do número de threads

4.3 - Discussão

No total foram feitas 33 configurações e em todas elas foram mais rápidas que a versão sequencial, com speedups que podem ser visto na tabela 2, a eficiência cai de forma consistente com o aumento do número de threads, como o particionamento distribui um volume fixo de trabalho entre um número crescente de processadores, a fração desse trabalho que cabe a cada um diminui, enquanto o custo fixo de coordenar a distribuição (abrir a região paralela, repartir as tarefas) não diminui na mesma proporção.

O custo computacional por ponto da malha é uniforme todo ponto executa exatamente a mesma sequência de operações do esquema de diferenças finitas, exatamente o cenário que a definição de balanceamento estático aponta como ideal, onde o tempo de computação por item constante é previsível de antemão. Isso explica por que static, que particiona o trabalho em blocos uma única vez e sem nenhuma coordenação em tempo de

execução, foi competitivo ou o melhor em várias configurações (por exemplo, chunk 8 com 4 threads: 0,299 s).

Já dynamic e guided implementam balanceamento dinâmico, o mesmo princípio do saco de tarefas, em que cada thread devolve seu bloco à fila compartilhada e retira um novo assim que fica ociosa, mecanismo criado para compensar problemas irregulares. Aplicado a uma carga já uniforme, esse mecanismo não tem desequilíbrio nenhum para corrigir

Sem collapse, apenas a dimensão mais externa da malha é fatiada entre as threads; com collapse(2) ou collapse(3), as dimensões fundidas passam a formar um único espaço de iteração, uma decomposição de domínio mais fina, com mais fatias disponíveis para distribuir entre os processadores. Isso explica por que collapse(2) obteve os menores tempos com 2 e 4 threads, mais unidades de trabalho tornam a divisão entre os processadores mais uniforme, mesmo sob balanceamento estático, reduzindo a chance de sobra de trabalho desigual entre eles.

5 - Conclusão

Foi implementada uma simulação da difusão viscosa em 3D por diferenças finitas, validada pela visualização do campo, em que uma perturbação inicial localizada se espalha suavemente e perde amplitude ao longo do tempo (Figura 2). A versão paralela com OpenMP foi avaliada em 11 variantes (schedule static, dynamic e guided com chunksize de 2, 4 e 8, e collapse(2) e collapse(3)) pois o laço principal possui 3 laços aninhados além disso também houve variação na quantidade de threads com 2, 4 e 8 threads, e todas reproduziram a mesma soma de u da versão sequencial.

Os speedups variaram entre 1,54x e 4,20x, quanto às cláusulas, o collapse(2) foi o mais rápido com 2 e 4 threads, seguido de guided com chunksize 2 e 2 threads. já com 4 threads o collapse(3) e o static com chunksize 8 tiveram destaque, já com 8 threads o dynamic com chunksize 2 foi o mais rápido. O logo o experimento nos mostra que variando a quantidade de carga entre as threads e reduzindo o desbalanceamento conseguimos obter uma melhor eficiência ao utilizar o multithreading.